

Herald



Herald — Fixed

Field	Value
Revision	15
Created	2026-05-20
Last modified	2026-05-27
Status	active
Status summary	r15 atomically migrates (§11.4.19) the completed GAP-3 §11.4.85 stress+chaos suite — HRD-122..HRD-130 (Task→Completed) — and the HRD-132 Runner idempotency-window fix (Bug→Fixed): 10 rows Issues→Fixed. GAP-3 commits 02d272d→d2a60d3 (HRD-122 commons/stresschaos/scaffold, HRD-123 Gin /v1/events, HRD-124 /v1/compliance+/v1/safety_state, HRD-125 Runner — surfaced HRD-132, HRD-126 pherald listen inbound, HRD-127 claude_code dispatch, HRD-128 container/resource §12.6-safe, HRD-129 e2e E81-E88, HRD-130 paired §1.1 mutation gate); HRD-132 fix 354b883 (claim-before-dispatch at Runner Stage 2 — INSERT events_processed ... ON CONFLICT DO NOTHING is the authoritative exactly-once dispatch gate; latent runner .go:132 CachedRcpt.WasReplay race closed; HRD-125 assertion upgraded to sends==1)+ wave3-M4-retire 4f45f26. Full scripts/ e2e_bluff_hunt.sh 78 PASS / 0 FAIL; HRD-130 gate 6 PASS / 0 FAIL (5 load-bearing mutations); HRD-132 evidence docs/qa/HRD-125-stress-chaos-2026-05-27T131251-gap3/ (dispatch_exactly_once=1). HRD-131 (SQLite SSOT migration Phase 1) remains OPEN. Prior r14 captured Wave 6 (pherald inbound runtime + Claude Code headless bridge — code-doc closure). HRD-100 closed atomically with 12 Wave-6 commit SHAs + 8 new e2e invariants E63-E70 (currently SKIP-with-documented-reason — converting to PASS once operator runs the live closed-loop at T10b and commits docs/qa/HRD-100/). Tag v0.4.0 DEFERRED to T13b (operator-supplied live evidence per §107.x — code-doc closure ships in this commit; runtime evidence lands separately).** Wave 6 surfaces: new Cobra subcommand pherald listen (Telegram getUpdates long-poll + OnText/OnPhoto/OnDocument/OnVoice + bot self-filter + sha256 content-addressed attachment download); Opus model pin in spawned claude argv (--model claude-opus-4-7); envelope pre-text — verbatim operator wording "We have received new message from our communication channel" preceding the <<<HERALD-DISPATCH-v1>>> structured block; <<<HERALD-REPLY>>> parser with action routing

Field	Value
	(reply / issue.open / event.emit); tgram.Adapter.SendReply wiring reply_to_message_id; --qa-out-dir JSONL journaling per §107.x. Wave 6 mutation gate (tests/test_wave6_mutation_meta.sh) lands 3 paired hermetic mutations (M1 blank bot self-filter → echo-loop signature; M2 Opus → Sonnet model swap → E67 FAIL; M3 drop opts .ReplyTo in SendReply → E68 FAIL). Prior r13 captured Wave 4b (TOON application / toon content negotiation; tag v0.3.0). r12 captured Wave 4a (HTTP/3 + Brotli + Alt-Svc + TLS 1.3; tag v0.2.0). r11 closed HRD-016 (Wave 3b pherald Runner live). r10 closed HRD-011 (Telegram live).
Issues	see Issues.md — r16 (Issues.md) captures the HRD-122..HRD-130 + HRD-132 Issues→Fixed atomic migration recorded in this r15.
Issues summary	HRD-008/-015/-018 (in_progress) + HRD-019..HRD-027 + HRD-029..HRD-056 + HRD-081 + HRD-085..HRD-090 + HRD-131 still open (40 items).
Fixed	HRD-001..HRD-007, HRD-009, HRD-009b, HRD-010, HRD-011, HRD-012, HRD-013, HRD-014, HRD-016, HRD-017, HRD-028, HRD-080, HRD-092, HRD-093, HRD-094, HRD-095, HRD-096, HRD-097, HRD-098, HRD-099, HRD-100, HRD-110, HRD-111, HRD-112, HRD-113, HRD-114, HRD-122, HRD-123, HRD-124, HRD-125, HRD-126, HRD-127, HRD-128, HRD-129, HRD-130, HRD-132 (and HRD-018 partial — M1 + M2 components landed)
Fixed summary	GAP-3 §11.4.85 stress+chaos suite (HRD-122..HRD-130 Completed) + HRD-132 Runner idempotency-window fix (claim-before-dispatch, exactly-once dispatch sends==1); spec V1→V3 r9; Go module foundation + Foundation M1/M2/M3; HRD-010 commons_storage live wiring; HRD-011 Telegram live (live message_id=5 evidence 2026-05-22); HRD-012 Claude Code dispatcher live; HRD-016 pherald /v1/events Runner wiring (Wave 3b live — 7-stage pipeline + e2e E37-E42); universal §11.4.73 + §11.4.74 mandates propagated; I6 gate refined; Wave 2 flavor scaffolds (HRD-092..097) closed atomically; Wave 3a substrate (HRD-099 commons_auth JWT verifier) + 2 live REST routes (HRD-028, HRD-098) closed atomically.
Continuation see CONTINUATION.md.	

Table of contents

- [Recently fixed](#)

Recently fixed

ID	Type	Criticality	Title
HRD-018	task	high	commons_constitution Go package: Evaluator interface + 12 event-class e + constitution_state/constitution_bindings migrations + bundle captureer + mode-ladder runtime config + constitution_audit write-through (th gap) . The Evaluator framework, emit helpers, bundle-hash, mode-ladder, state state.Postgres, ladder.Memory/ladder.Postgres, and migrations 000006/000007/000008 were all coded in r1. v1.0.0 Batch A closed the remaining PASS-bluff: migration 000006 defined constitution_audit but NO code e row — Runner.Run set RunOutcome.Audited=true with nothing persist an AuditStore interface (audit.go) + state.MemoryAudit (append-on state.PostgresAudit (RLS-guarded INSERT-only into constitution_ append-only is enforced by the migration's no-update/no-delete RLS policies). Ru now writes a durable audit row for every CHANGED transition in ModeWarn/Mo

ID

Type Criticality Title

HRD-026 task middle

HRD-027 task middle

HRD-132 bug high

HRD-130 task high

recording old/new decision, old/new digest, bundle hash, mode-at-emission, and -
 ModeEnforce — the EXACT emitted event ID (new IDemitter optional interf
 PolicyViolationID/PolicyClearedID; emit returns the generated even
 NewRunner gained a required AuditStore arg (nil = hard error). §11.4.92 multi-pa
 TDD RED (compile-fail before backends) → GREEN, -race -count=3 dete
 Pass-5 real DB-level SELECT shows enforce rows carry emitted_event_id, warn
 Constitution-bundle hash captureer — computes SHA-256 of rendered Constituti
 evaluation time; persists on every emitted event for replayability. The captureer (l
 Capture/CaptureBytes/Captureer) was coded in r1 and the hash already
 emitted-event metadata (bundle_hash hex). v1.0.0 Batch A completed the pers
 the 32-byte bundle_hash is now stored on every constitution_state ro
 every constitution_audit row (verified by the DB-level SELECT ...
 encode(bundle_hash, 'hex') in the evidence dir showing a 64-hex SHA-
 every audit row), closing the §42.1.3 replay-correlation requirement.

Mode-ladder runtime config (constitution_bindings table + admin REST
 to flip allow/warn/enforce per binding per tenant without redeploy). The Model a
 interface + ladder.Memory/ladder.Postgres (reading
 constitution_bindings at evaluation time) were coded in r1. v1.0.0 Batch
 the admin REST surface: cherald/internal/modes — GET /v1/compl
 modes (list bindings + unbound-default), GET /v1/compliance/modes/:
 (one binding's effective mode, enforce default), PUT /v1/compliance/
 modes/:rule_id (flip allow/warn/enforce; records the operator sub claim in
 mutated_by). JWT-gated via the existing commons_auth.GinMiddleware
 mounted in cherald serve (a single shared PG pool backs both compliance + m
 anti-bluff: 401 no-auth / 401 bad-tenant / 400 bad-mode (loud, never a silent defa
 bearing proof: a PUT flip to warn is reflected by a subsequent GET AND by lad
 on the evaluation hot path — runtime config, no redeploy.

§11.4.85/§32.1 Runner idempotency-window FIX — concurrent same-key replay
 duplicate-dispatches a notification (FINDING from HRD-125 stress/chaos). Root
 idempotency.go:64-72 treated a Redis-SETNX-miss-with-no-PG-row as F
 §32.1 "Redis-lies-PG-truths" design) while the events_processed archive ro
 written at Stage 7 (OutcomeRecorder.Process) — so concurrent same-key
 arriving in the Stage-2→Stage-7 window ALL dispatched (observed 2-4 of a 100
 archival exactly-once via PG UNIQUE + ON CONFLICT DO NOTHING, but cl
 dispatch only bounded = duplicate-alert delivery under load). Fix: claim the idem
 atomically BEFORE dispatch — INSERT events_processed ... ON CON
 NOTHING moved to Stage 2 (the PG row, serialized by PRIMARY KEY(tenant_i
 idempotency_key), is now the AUTHORITATIVE exactly-once dispatch gate; 0-r
 affected = duplicate → no dispatch), preserving the nil-Redis PG-only degrade. A
 the latent runner.go:132 CachedRcpt.WasReplay data race (mutated a
 shared *Receipt). The §11.4.85 HRD-125 stress test was upgraded to assert se
 under the same 1000× shared-key flood — the stronger assertion HRD-125 delibe
 NOT make pre-fix (§11.4.4 test-interrupt-on-discovery). §11.4.92 multi-pass: Pas
 independent -race -count=3 runner + http + storage green; Pass-5 sends=
 true (RED proved before fix). wave3 mutation-gate M4 retired (4f45f26) — St
 now redundant, M4's blast-radius moved into the Stage-2 claim.

§11.4.85 / §1.1 paired mutation gate for the GAP-3 stress+chaos suite — tests.
 test_stress_chaos_mutation_meta.sh (GAP-3 plan §5 / §6 T9). Prov
 hermetic stress/chaos detector is genuinely LOAD-BEARING via the canonical t
 test_wave6.5_mutation_meta.sh idiom: .git/MUTATION_IN_PRO
 lockfile, dirty-tree + pre-existing-residue refusal pre-flights, run_paired (appl
 mutation → assert marker landed → run detector EXPECT-FAIL → git check

ID

Type Criticality Title

HRD-129 task high

→ byte-for-byte assert_restored vs HEAD → re-run EXPECT-PASS), and check_quiescence scan asserting no MUTATED SC-M* markers leaked (§11.4.85 paired mutations, each against a hermetic detector: SC-M1 drop ctx from claude_dispatch.go exec.CommandContext→exec.Command → timeout-cancel longer cancels the 30s sleep → watchdog FAIL; SC-M2 swallow the claude_code_parseReply decode error → truncated-reply test returns nil → FAIL; SC-M3 inbound extractReplyToMessageID default: panic instead of return → degrade test panics → FAIL; SC-M4 mmap events mapErrorToStatus event_parser: 400→500 → input-corruption test sees 5xx → FAIL; SC-M5 commons_storage applyMigration disk-full write error → ENOSPC reports silent-swallow guard FAIL. Runs ONE-AT-A-TIME, FOREGROUND (§107.y + 2026-05-27 concurrent-gate-residue memo). Conductor-executed result: 6 PASS / mutations load-bearing).

§11.4.85 stress+chaos suite wired into the canonical anti-bluff harness scripts/e2e_bluff_hunt.sh as new invariants E81-E88 (GAP-3 plan §4 / §6 T8) — ADDITIVE-ONLY append after E80 (E1-E80 logic untouched). Each invariant R REAL hermetic Go stress/chaos test via the existing check helper AND cites the evidence artefact under docs/qa/<run-id>/stress_chaos/ as its §11.4.85 positive-evidence anchor through a new sc_anchor helper that greps a SPECIFIC bearing value (present-but-empty artefact FAILs as a §11.4 PASS-bluff; absent ev SKIPs-with-reason). Mapping: E81 runner exactly-once-archive + bounded dispatch (HRD-125); E82 /v1/events input-corruption + duplicate-key + auth-storm (HRD-126); E83 /v1/compliance fail-loud + duplicate-key (HRD-127); E84 /v1/safety_state concurrent-mutation (HRD-124); E85 pherald 1 inbound malformed-Raw panic-free + extract-degrade (HRD-126); E86 claude_code death + timeout-cancel + truncated-reply (HRD-127); E87 disk-full tagged-error 1 (HRD-128); E88 §12.6 host-mem headroom (HRD-128). All eight hermetic; live pause / real-disk-fill / container-OOM variants are SKIP-with-reason inside the G (HERALD_STRESS_LIVE_*=1 gated). Verified bash -n clean; E81-E88 block PASS deterministic; header bumped E1..E80→E1..E88. Full scripts/e2e_bluff_hunt.sh reports 78 PASS / 0 FAIL (was 60; +18).

HRD-128 task high

§11.4.85 container-orchestrated / resource-exhaustion stress+chaos tests (GAP-3 plan §6 / §6 T7 + §7 host-safety) — commons_storage/resource_stress_chaos_test.go (hermetic core) + tests/test_resource_stress_chaos.sh (LIVE container harness). HERMETIC deterministic): (1) disk-full error propagation — a fake database.Database.Syscall.ENOSPC on the migration-UP exec, driving the REAL RunMigration applyMigration write path; asserts real write path exercised, error NOT swallowed tagged commons_storage: apply + exec up, errors.Is(err, syscall.ENOSPC) true, 0 migrations reported applied. (2) §12.6 host-mem headroom HostMemHeadroom() pre/post a 25× workload shows used_fraction_decreased (host needle did not move) = §12.6 compliance evidence. OPT-IN (deterministic) without env): real bounded-scratch disk fill HARD-capped 64 MiB, gated HERALD_STRESS_LIVE_DISK=1. LIVE container scenarios SKIP-with-reason (host-safety-gated): container OOM-confinement + connection-churn against a -memory-bounded PG, gated HERALD_STRESS_LIVE_OOM=1 + a §12.6 pre-flight headroom gate that REFUSES to add pressure when host used-fraction ≥ 60% (host ~74% this run → honest SKIP). §12/§12.6 honoured: NO host-OOM, NO real-host process killed. go test -race -count=3 green (PASS×3/PASS×3) go vet clean.

HRD-127 task high

ID

Type Criticality Title

HRD-126 task high

§11.4.85 `claude_code` dispatch stress+chaos tests (GAP-3 plan §1 row 5 / §6 T6) — `commons_messaging/dispatch/claude_code/dispatch_stress_chaos_test.go` (in-package, exercises unexported `bootstrapSession/parseReply`) + 5 committed hermetic shims `testdata/claude-*.sh`. Per §11.4.27 only the EXTERNAL `claude` CLI is faked via `th` `New(binaryPath,...)` seam; NO production source edited, NO new dep, NO new invoked. STRESS: (1) $N=24 \times 25=600$ concurrent `Dispatch` (`--resume` path) every reply really parsed, every call resumed the SAME pre-seeded UUID, anchored, drifted, $p50=62.9\text{ms}/p95=96.4\text{ms}$; (2) $N=16$ concurrent COLD-START bootstrap succeed, exactly one anchor UUID persists, `-race` clean. CHAOS: (a) `process-d` `137 + self-kill -KILL $$` → tagged error, no hang (watchdog), partial stdout accepted; (b) timeout — 30s-sleep shim vs 800ms ctx → cancellation fires, `Dispatch` ~803ms not 30s; (c) truncated `<<<HERALD-REPLY>>>` → explicit decode `r` JSON error, never silent partial-accept. FINDING-1: cold-start concurrent bootstrap writer-wins by design (honest bound asserted, not `bootstrap_count==1` blue). FINDING-2: timeout-cancellation latency depends on `claude` being a single process (hardening direction = process-group kill, flagged for dispatcher owner). `go test -count=3` deterministically green $\times 3$.

§11.4.85 `pherald` listen inbound-path stress+chaos tests (GAP-3 plan §1 row 6) — `pherald/internal/inbound/stress_chaos_test.go` (in-package, exercises unexported `extractReplyToMessageID`) + `pherald/cmd/pherald/listen_stress_chaos_test.go` (drives the REAL `runListen` loop). Per §11.4.27 only the channel `Subscriber` + CC `CodeDispatcher` boundaries are faked; the `action-routing` switch + `runListen` fan-in + clean-shutdown run unmodified. STRESS: $N=16 \times 50=800$ concurrent `Dispatcher.Handle` — 0 errors, exactly-once reply (800/800), 0 goroutine leak, $p50=0.41\text{ms}$ ~25k dispatch/s; (2) 400-event burst through `runListen` — nothing dropped, clean shutdown on cancel ($<3\text{s}$), no leak. CHAOS: corrupt Raw payloads (nil map, missing/nil/string/bool/slice/map `message_id`) → `extractReplyToMessageID` degrades to (0,err); valid `int/int64/int32/float64` decode; 64-dispatch corrupt-Raw flood panic-free (recover-guard), degraded reply (b) CC subprocess fault (killed/truncated/marker-less/empty/unknown-action) → `inbound`: error, no reply leak, no hang; (c) once-only side-effect — 1000 redelivered same `issue.open` collapse to exactly-ONE durable sink row. `go test -race -count=3` deterministically green $\times 3$ on both packages. NO production source edited, NO new dep.

HRD-125 task high

§11.4.85 `Runner` 7-stage pipeline stress+chaos tests (GAP-3 plan §1 row 3 / §6 T6) — `pherald/internal/runner/stress_chaos_test.go` exercising the `runner.Run` (only the external boundary — PG `events_processed` store, channel `Redis` SETNX seam — faked per §11.4.27; orchestrator + 7 stages run unmodified). STRESS: $N=16$ goroutines $\times 40$ iters concurrent same-key replay + fresh keys, `-race` clean. (a) 1000 $\times$ duplicate flood @ 50 parallel under live atomic `Redis` SETNX; (a') same nil-`Redis` PG-fallback degrade; (b) hermetic PG-deadlock (SQLSTATE 40P01) in the `events_processed` `Insert`. SURFACED HRD-132: dispatch exactly-once was NOT guaranteed under concurrent replay (`events_processed` archived at Stage 7; observed duplicate dispatches of a 1000 $\times$ flood); tests asserted the honest code-true contract (exactly-once + bounded dispatch) pre-fix, then upgraded to `sends==1` once HRD-132 landed the Stage-2 claim. `go test -race -count=1` green.

HRD-124 task high

§11.4.85 `Gin` `/v1/compliance` (`cherald`) + `/v1/safety_state` (`sherald`) stress+chaos tests (GAP-3 plan §1 row 2 / §6 T3) — `cherald/internal/compliance/compliance_stress_chaos_test.go` + `sherald/internal/safety/safety_stress_chaos_test.go`, each driving the REAL flavor over a REAL in-process `httpTest` server dialed by `net/http.Client` through

ID

Type Criticality Title

HRD-123 task high

production Gin chain (`cli.TOONMiddleware` → `auth` → `handler`). Only the external boundary faked per §11.4.27; NO production source edited, NO new dep. STRESS (flavor): $N=10 \times 200=2000$ GETs, -race clean — 0 errors, 0 5xx, all 200 (compliance req/s, safety_state ~16k req/s). Content-negotiation correctness under concurrency: request alternates `Accept: application/toon` vs `application/json`; Content-Type match to its OWN Accept header (0 codec-bleed), every toon body is valid wire bytes. CHAOS: (a) compliance PG-drop (`errStore`) → 2000/2000 fail-loads; naming the failed dependency, 0 fabricated 2xx (a swallowed-error 200 {result: "error"} would falsely tell an operator "no violations" — the §107 mode this gate forbids); (b) has no hot-path DB by design so the code-true analogue (concurrent state-mutation under load, -race-clean RWMutex) is exercised instead; (c) auth storm — 1000 random tokens through the REAL `commons_auth.GinMiddleware+HMAC(HS256)` → 1000×401, 0 bypass. Verified `go test -race -count=3` green ×3.

§11.4.85 Gin /v1/events stress+chaos tests (GAP-3 plan §1 row 1 / §6 T2) — `internal/http/events_stress_chaos_test.go` driving the REAL `events` handler (`EventsHandler` → `runner.Run`, all 7 stages) over a REAL `httptest` server dialed by `net/http.Client` through the production Gin chain (`cli.TOONMiddleware` + `auth` gate). Only the external boundary (PG/Redis/cross-worker `runner.NewFakeRunner`) faked per §11.4.27. STRESS: $N=12 \times 200=2400$ C, 1200 POSTs, -race clean — 0 errors, 0 5xx, all 202; $p50=0.706\text{ms} \sim 13,322$ req/s. CHAOS: (a) duplicate-key-under-load (1200 POSTs, per-worker keys) → graceful degrade, 0 5xx; (c) input-corruption (8 MiB random, truncated JSON, non-UTF8, JSON array, missing fields, garbage, empty) × 5 → every body a tagged `event_parser: 4xx`, 0 panic, 0 5xx; (d) auth storm — 1000 random tokens through the REAL `commons_auth.GinMiddleware+HMAC(HS256)` → 1000×401, 0 bypass; (e) PG-drop → SKIP-with-reason (proven hermetically at HRD-125). FINDING (→ HRD-125): HTTP layer: a SHARED-key cross-worker concurrent replay flood triggers the `runner.go:132 CachedRcpt.WasReplay` data race (fixed by HRD-132). re-captured as `all_malformed_rejected_no_5xx` (the §11.4.50 transport layer of 8-MiB-body is valid).

HRD-122 task middle

§11.4.85 stress+chaos shared scaffold — Go-native `commons/stresschaos` package (GAP-3 plan §2 / §6 T1). Provides: a bounded concurrent load-runner (`Runner`) with N workers × M iterations of a closure via `stdlib sync.WaitGroup`, no new dependencies; nearest-rank percentile math (`Percentiles` → {`p50`, `p95`, `p99`, `max`, `min`}, emitted as `latency.json+latency_histogram.csv`); an evidence-dir v1 creator (`creating docs/qa/<run-id>/stress_chaos/<surface>/`); and a best-effort memory headroom probe (`HostMemHeadroom` — `vm_stat+sysctl hw.meminfo` on darwin, `/proc/meminfo` on linux) for the §12.6 60%-ceiling proof (degrades to `unavailable`, never a hard failure). Self-tested: 8 unit tests (percentile correctness on known 1..100ms dataset, evidence-dir creation, bounded-goroutine proof, host-memory probe effort, §12.6 ceiling predicate) PASS under `go test -race -count=1`. Low effort layer per CLAUDE.md layering (commons L0, importable by every flavor).

HRD-114 task high

Wave 7 T5 — multi-channel pherald `listen` (`HERALD_CHANNELS fan-in` → `Subscribe` goroutines → 1 dispatcher). `pherald/internal/inbound/dispatcher.go` renames the Wave-6 `TgramReplier` interface (Telegram-native `chatID/int replyToID` signature) → generic `inbound.Replier` with `SendReply(ctx, recipient commons.Recipient, body string, replyToID string, attachments []commons.Attachment) (string, error)` — recipient + string ids let any channel satisfy it. `Config.TgramReplier` → `Config.Reply`; `NewDispatcher` nil-check + `Dispatcher.reply` field; `Handle "reply" branch` + both `fastPath` branches all migrated to build a `commons.Recipient{Channel, ChannelUserID}` +

ID

Type Criticality Title

strconv.Itoa(replyToID) (" " when 0). pherald/cmd/pherald/listenConfig.Subscriber (singular) → Subscribers
map[string]func(ctx, InboundHandler) error; listenConfig.inbound.TgramReplier → inbound.Replier; new loadEnabledChannels parses HERALD_CHANNELS (comma-split, trim, dedupe; unset/empty → ["tgram"] Wave-6 single-channel behaviour preserved); new perChannelConfig(name, namespace, env (HERALD_TGRAM_BOT_TOKEN/HERALD_TGRAM_CHAT_ID/HERALD_SLACK_BOT_TOKEN/_APP_TOKEN/_CHANNEL_ID) → channelConfig (fail-load on missing required creds); loadListenConfigFromEnv resolves enabled channel via the channels.New(name, cfg) registry (T2), builds Subscribers[name]=ch.Subscribe + a channelReplier{ch} shim channels.Channel.SendReplyGeneric → inbound.Replier.SendReply (the two intentionally-differently-named generic methods bridged) per channel, and them in a channelRouter that dispatches each reply to the adapter for recipient.Channel (fail-load no replier for channel %q on unknown channel never silently dropped). runListen fans in one goroutine per subscriber (manual sync.WaitGroup + buffered error channel; no new dep — errgroup-equivalent) all feeding ONE dispatcher; the first non-cancel subscriber error cancels siblings (T11 chaos fail-load). tgram is blank-imported in listen.go for its init() registration. journal.go journalingReplier migrated to wrap inbound.Replier (records channel+chat_id+reply_to_message_id strings; JSONL kind kept tgram.send_reply for transcript back-compat). M resolution (Wave 7 plan Step 4): inbound.Replier.SendReply keeps the SendReply name (the inbound package's own, independent of channels.Channel.SendReplyGeneric); production wiring bridges via channelReplier shim so the journal decorator + dispatcher both stay channel-agnostic §107 evidence: new TestRunListenMultiChannelFanIn (2 stub subscribers tgram+slack each publish 1 event) asserts BOTH messages dispatched AND both recorded (seen["ack (fake)"] >= 2) — NOT "both goroutines started"; proven load-bearing (dropping the fan-in loop → seen == map[] → FAIL). E single-channel TestListenWiresHandlerAndExitsOnCancel + TestRunListenRejectsBadConfig migrated to the one-entry Subscriber still PASS. recordingReplier/stubReplierJournal stubs (dispatcher listen_test/journal_test) migrated to the generic signature; numeric chat_id/reply_id preserved via decode.go build ./pherald/... + go test -race -count=1 ./pherald/... ./commons_messaging/... all green; go clean; §107.y quiescence clean.

Wave 7 T4 — generalize the bot self-filter via BotSelfIdentity (channel-agnostic) file commons_messaging/channels/selffilter.go adds the Raw-message constants (RawSenderIsBot/RawSenderIdentityKind/RawSenderIdentityStampSender(raw map[string]any, isBot bool, kind IdentityKind, value string) (records the sender's native identity into ev.Raw; nil-map no IsSelfEcho(ev commons.InboundEvent, self SelfIdentity) §32.9 anti-echo-loop guarantee made channel-agnostic — compares the right native IdentityKind: username for Telegram, user_id for Slack, address for Scope is deliberately narrow: a DIFFERENT bot is KEPT (multi-bot collaboration subscriber traffic — the §107 anchor that the filter is not over-broad); a human is empty self.Value never echoes (conservative KEEP → surfaces as duplicate silent loss; the real guard is Subscribe refusing to boot on empty self). commons_messaging/channels/tgram/subscribe.go migrated: cap self := channels.SelfIdentity{Kind: channels.IdentityUser, Value: bot.Me.Username} once at Subscribe boot (keeping the existing echo

HRD-113 task high

ID

Type Criticality Title

HRD-112 task middle

username boot-refusal echo-loop guard) and routes ALL FOUR live handlers (OnPhoto/OnDocument/OnVoice) through a new stampAndIsSelfEcho helper that stamps the Telegram sender's IsBot+@username into ev. delegates to channels.IsSelfEcho — replacing the four Wave-6 shouldDropBotSelf(msg, selfUsername) early-returns. The Wave-6 shouldDropBotSelf func + its TestSubscribeBotSelfFilter + SelfFilterForTest (export_test.go) are PRESERVED BYTE-IDENTICAL M1 paired-mutation gate (tests/test_wave6_mutation_meta.sh) and exact-text perl -i -0pe regex on shouldDropBotSelf's body, so keeping identical keeps M1 load-bearing (verified: the M1 regex still matches + applies its marker cleanly post-migration). The live path and the gate now express the SAME invariant — production on the generalized IsSelfEcho, the gate on the tgram-§107 evidence: TestIsSelfEchoUsername proves all three username cases DROPPED, different-bot KEPT, human KEPT; TestIsSelfEchoUserID proves Slack user_id kind (matching bot id DROPPED, different id KEPT); TestIsSelfEchoEmptySelfNeverEchoes proves empty-self KEEP; the helper builds events via the production StampSender (DRY — the test can never the wire-key contract); TestSubscribeBotSelfFilter still PASSES (Wave preserved); go test -race -count=1 ./commons_messaging/... pherald/... all green; go vet clean. Live runtime evidence (real Telegram identity + own-message drop in the closed loop) lands via Wave 7 T9 e2e invariant Wave 7 T3 — per-channel inbox subdirs ~/ .herald/inbox/<channel>/<sha256>.<ext>. New file commons_messaging/channels/inbox. channel-agnostic InboxDir(channel string) (string, error) (res ~/ .herald/inbox/<channel>/, MkdirAll 0700, empty-channel error) + WriteContentAddressed(channel, mime string, r io.Reader) (path, sha256Hex string, err error) (streams r into <dir>/<sha256>.<ext> via io.MultiWriter(tmp, sha256.New()) — never the full payload — then atomic temp+rename; idempotent: if the content-addressed already exists the .part temp is dropped and the existing path returned unchanged + MimeToExt(mime string) (the Wave-6 tgram-private mimeToExt switch VERBATIM to package level so every adapter shares one map). commons_messaging/channels/tgram/attachments.go DownloadAttachment now does Telegram-specific bot.File(&telebot.File{FileID}) fetch then deleg stream-hash-rename to channels.WriteContentAddressed(string(common.ChannelTo mime, rc) — so tgram attachments land under ~/ .herald/inbox/tgram channel subdir, was the flat ~/ .herald/inbox/ in Wave 6). The tgram-private mimeToExt + the crypto/sha256/encoding/hex/io/os/path/filepath were removed from attachments.go (the helper owns them now); the method (*Adapter).DownloadAttachment(T1) delegates to the package func un both forms route through the shared helper. §107 evidence: TestInboxDirIsPerChannel proves InboxDir("tgram") ends in . . tgram AND InboxDir("tgram") != InboxDir("slack") (no flat-in regression); TestWriteContentAddressedHashesAndIsIdempotent three §107 bluff classes on real on-disk state — (a) path shape under inbox/sl <sha>.txt, (b) re-read on-disk bytes byte-equal the payload (sha in path == sha content), (c) 2nd write of same content returns same (path, sum) and leaves zero residue (real os.ReadDir scan); the existing Wave-6 TestDownloadAttachmentContentAddressed (updated to the /tgram segment) still PASSES end-to-end against a httptest Bot API, proving the route is behaviour-preserving. go test -race -count=1 ./

ID

Type Criticality Title

HRD-111 task high

commons_messaging/... ./pherald/... ./qaherald/... all gro
assumption regressions); go vet clean. Live runtime evidence (real Telegram at
landing under inbox/tgram/) lands via Wave 7 T9 e2e invariant E83.
Wave 7 T2 — channel registry (name→constructor) + init()-based registration
commons_messaging/channels/registry.go adds Register(name
Constructor)/New(name, Config) (Channel, error)/Names(
[]string, the ErrUnknownChannel sentinel, the channel-agnostic Config
(Channel/Token/AppToken/Target/BaseURL/Extra), and the Constr
func(Config) (Channel, error) type — a sync.RWMutex-guarded
map[string]Constructor. Register panics on duplicate (a typo in two
init() would silently shadow one — a §107 bluff class this framework forbids
qaherald scenario. Register pattern). New wraps ErrUnknownChannel
(fmt.Errorf("%w: ...")) for unknown names — callers (pherald lis
MUST surface it; a silent no-op channel is a §107 PASS-bluff. commons_mess
channels/tgram/tgram.go gains an init() that calls
channels.Register(string(common.ChannelTelegram), ...
constructor builds NewAdapterWithBaseURL(cfg.Token, cfg.Target
cfg.BaseURL) when cfg.BaseURL != "" (http test seam) else
NewWithCreds(cfg.Token, cfg.Target) — so a blank import self-reg
adapter. §107 evidence: TestRegistryResolvesRegisteredChannel p
actually constructs + returns a working Channel (c.Name()=="fake-rt",
TestRegistryUnknownChannelErrors proves errors.Is(err,
ErrUnknownChannel) for unknown names; TestRegistryDuplicateP
proves the duplicate-Register panic; TestTgramRegisteredViaInit(k
_ ".../tgram") proves the init() actually registered tgram in Names()
op — verified failing before the init() landed: Names()=[dup-rt fake-
tgram, PASS after). The shared fakeChannel test stub uses SendReplyGene
T1-resolved interface method name, NOT SendReply). go test -race ./
commons_messaging/... green (tgram regression-free); all 16 workspace m
build; go vet ./commons_messaging/... clean. Live runtime evidence
channel resolved by name + round-trip) lands via Wave 7 T9 e2e invariants E81/E
Wave 7 T1 — extract commons_messaging/channels.Channel interface
refactor). New package commons_messaging/channels/ defines Channe
the §11.0 commons.Channel — Name/Capabilities/Send/Subscribe/HealthCh
the three inbound-runtime methods Telegram implicitly defined: SendReplyGene
string-typed reply, BotSelfIdentity returning SelfIdentity{Kind, Va
DownloadAttachment), the SelfIdentity value type, and the Identit
constants (IdentityUsername/IdentityUserID/IdentityAddress).
*tgram.Adapter gains BotSelfIdentity (getMe via ensureBot; empty-u
§107 echo-loop-hazard error), SendReplyGeneric (string→int64/int adapter
native Wave-6 SendReply), and a method-form DownloadAttachment (rou
package-level func through ensureBot's live bot) — so var _ channels.Cha
(*tgram.Adapter)(nil) compiles. Naming divergence from plan Step 4 to
+ documented in channel.go package doc: the interface reply method is name
SendReplyGeneric (not SendReply) so tgram satisfies it WITHOUT touch
native, §107-mutation-gate-anchored int64/int SendReply (whose migration to
signature is T5's scope) — keeping T1 a pure refactor with zero cross-package ch
evidence: var _ channels.Channel = (*tgram.Adapter)(nil) co
assertion in channel_test.go (a renamed method / drifted signature breaks t
TestTgramSatisfiesChannel + TestSelfIdentityType PASS; full
-race ./commons_messaging/... green (tgram regression-free — pure-
proof); all 16 workspace modules build; go vet ./commons_messaging/

HRD-110 task high

ID

Type Criticality Title

HRD-101 task high

Live runtime evidence (real getMe self-identity + reply round-trip) lands via Wave invariant E81.

Wave 6.5 ticket-lifecycle layer — §32.6 deterministic classifier (Bug:/Issue:/Implementation:/Investigation:/Query:/Question:/Help:/Status:/Continue:/Done:/Resolve:/Reopen:/Override: prefix routing with Type+Criticality+Confidence scores; natural-language no-prefix → qa fallback) + DocsIssueOpener (§32.9 — real docs/Issues.md HRD-NNN allow concurrent-safe via lockfile) + RunnerEventEmitter (§33.7 — direct runner .Run CloudEvent dispatch, no HTTP loopback) + Help:/Status:/Continue: fast handlers (bypass Claude Code, sub-second latency — S2 proved 0.74s, no CC bootstrap overhead) + Done:/Reopen: atomic Issues[?]Fixed migration (rollback-via-sha256 tgram .SendReply outbound attachment fan-out (multipart dispatch) + wiring into pherald listen. §107.x live evidence (docs/qa/HRD-101-lifecycle-2026-05-23T03-16-17-w6.5live/): S1 natural-language real Opus CC dispatch ~31s incl. bootstrap, threaded reply (wire-byte proof inbound message_id=30 → reply sent_message_id=31 with reply_to_message_id=30); S2 Help: fast-path — classifier Type:help Confidence:1, CC dispatch ABSENT (event absence + 0.74s latency prove no BuiltinHelp catalogue text returned (reply_to_message_id=32 → sent_message_id=33); transcript.jsonl captures both directions of all four messages query, Help:, Status: also exercised live). Reproducible test harness tests/test_wave6.5_lifecycle.sh covers 15 scenarios (Bug:/Task:/Query: + a Done:/Reopen: + classifier edge cases). Spec V3 r13: §32.6/§32.9/§33.6/§33.7 successfully landed. Tag v0.5.0.

HRD-100 task high

Wave 6 — pherald inbound runtime + Claude Code headless bridge (code-doc close Cobra subcommand pherald listen runs the closed-loop runtime: Telegram getUpdates long-poll (OnText/OnPhoto/OnDocument/OnVoice) + bot self-filter (bot.Me.Username anti-echo guard captured at Subscribe construction; refuse getMe returned no username) + sha256 content-addressed attachment download ~/ .herald/inbox/<sha256>.<ext> (idempotent — duplicate download allowed write) + Claude Code dispatch with Opus pinned in the spawned argv (claude claude-opus-4-7 --resume <UUID> --print <envelope>) + envelope pre-text — verbatim operator wording ("We have received new message through communication channel <channel-name>. <classification> sentence>. <attachment list>") preceding the <<<HERALD-DISPATCH v1>>> structured block + <<<HERALD-REPLY>>> parser with action routing default / issue.open / event.emit) + tgram.Adapter.SendReply(chatID, text, replyToID, attachments) wiring reply_to_message_id from the original message_id. --qa-out-dir <path> flag journals every outbound event as JSONL per §107.x. Closing 12 atomic Wave-6 commits T1=ac (HERALD_PROJECT_NAME resolver + .env.example), T2=0d55274 (Opus message T3=14626f7 (envelope pre-text), T4=b4018a3 (bot self-filter), T5=d9156f1 OnDocument/OnVoice + sha256 download), T6=a22a054 (pherald listen subcommand), T7=a3b238a (pherald/internal/inbound/Dispatcher routing), T8=5f92f8a (tgram .SendReply with reply_to_message_id T9=f7913b8 (live closed-loop e2e script tests/test_wave6_live_loop T10a=520e0c6 (--qa-out-dir flag + JSONL journaling middleware), T11= (e2e_bluff_hunt E63-E70 invariants), T12=96c7c6b (Wave 6 mutation gate tests/test_wave6_mutation_meta.sh — 3 paired hermetic mutations). §107 evidence new e2e invariants E63-E70 land in this commit (all currently SKIP with documentation — E63 lifecycle / E64 bot self-filter / E65 attachment sha256 / E66 envelope pre-Opus argv / E68 reply_to_message_id wire-bytes / E69 action routing issues

ID

Type Criticality Title

HRD-016 task middle

HRD-011 task middle

HRD-099 task low

HRD-098 task middle

HRD-028 task low

HRD-092 task middle

HRD-093 task middle

live closed-loop). Conversion SKIP → PASS lands at T10b when the operator exports `HERALD_TGRAM_BOT_TOKEN + HERALD_TGRAM_CHAT_ID + HERALD_CL` and runs `bash tests/test_wave6_live_loop.sh --qa-out-dir`. HRD-100-<run-id>/. Tag v0.4.0 is DEFERRED to T13b (post-T10b live c §107.x) — code-doc closure ships here.

`pherald /v1/events` Runner wiring (REST API surface via Gin Gonic per spec §32 7-stage event-ingest pipeline lands as 7 concrete structs under `pherald /internal/runner/` (EventParser, IdempotencyChecker, TenantResolver, PolicyGate, SubscriberResolver, ChannelDispatcher, OutcomeRecorder) orchestrated by `runner.Runner.Run`. `pherald/internal/http/events.go` exposes HTTP handler; `pherald/cmd/pherald/{main,stubs}.go` lazily build JWT verifier inside the `serve` subcommand so other paths still work without `HERALD_PG_DSN/HERALD_AUTH_MODE`. HTTP contract: 202 Accepted + Re on fresh event, 200 + X-Herald-Replay: true on replay, 401 on missing/in 400 on parse failure. §107 evidence: 23/23 runner package tests green (per-stage integration scenarios: happy/duplicate/deny); 6 new e2e invariants E37-E42 (PG skipped when absent — honest SKIP-with-reason per §11.4.3); mutation gate `test_wave3_mutation_meta.sh` gains M2 (Duplicate=false unconditional breaks), M4 (skip events_processed.Insert → E38/E42 break). M3 SKIP-with-re Wave 3c deny-evaluator e2e. Commits T1..T10: eb9b2f4, 40ad60f, 2fdb7b 4c4af24, 3988114, 5468897, 6ea29a4, be71db1, c7b89a4, (this comm

Telegram channel adapter live integration — `telebot.v3` vendored (submodules `telebot`); `HealthCheck + Send + Subscribe + outbound_delivery_evidence` per live-wired with §107 bluff guards. Closed atomically with operator-session live e 2026-05-22: real bot delivery message_id=5 from @atmosphere_worker_ 2057253161 (validated via `getChat`) using the extended wizard CLI's `--bot` (env: `HERALD_TGRAM_BOT_TOKEN`) + `--chat-id` (validated via `getCh` persist) + `--non-interactive` flags. Token→getMe→getChat→sendMessage confirmed end-to-end in commit 140a2f1 session.

`commons_auth/` JWT verifier — HMAC + JWKS hybrid + Gin middleware + cla vendored Herald-internal per §11.4.74 catalogue-check no-match. Catalogue evid docs/catalogue-checks/HRD-099-commons-auth.md. Renumbered HRD-093 (Wave 3a executing-time anchor) in the post-Wave-3a doc-cleanup pas commits dbbea95..0cc6fad keep the old HRD-093 in their commit message

`sherald /v1/safety_state` live — process-local Aggregator + background m (15s tick) + Handler with JWT gate via `commons_auth.GinMiddleware`. Replaces 501-stub. §107 evidence: e2e E46 (no-auth → 401), E47 (valid HMAC → 200 + current_mem_percent>0 + last_destructive_op=null + uptime_seconds>=1 + oper Paired mutation M5 (zero-mem Aggregator) proves E47 catches the regression. E with-reason — destructive-op trigger awaits HRD-033 body.

`cherald /v1/compliance` live — paginated + filter-aware `constitution_` surface with JWT gate via `commons_auth.GinMiddleware`. Backed by `ConstitutionStore.ListQuery` extended with Since/Until/Offset (commit 6276437) evidence: e2e E43 (no-auth → 401), E44 (valid HMAC + empty tenant → 200 + page=1 + page_size=50). E45 SKIP-with-reason — cross-binary `cherald-reads-pl` deferred to Wave 3b (Runner not live yet).

`commons/cli/` shared CLI scaffold (Wave 2) — `NewRootCmd + VersionCmd + S` (with Middleware hook) + `StubCmd + healthz/readyz/metrics` handlers. Vendored internal per §11.4.74 catalogue-check no-match. As-built evidence: §44.M of spec docs/catalogue-checks/HRD-092-commons-cli.md.

ID	Type	Criticality	Title
			sherald flavor scaffold — System Herald binary serving :24793 with cli.ServeCmd stubs (HRD-033/034/040/046/056) + /v1/safety_state 501 stub → HRD-098.
HRD-094	task	middle	cherald flavor scaffold — Constitution Herald binary serving :24792 with cli.ServeCmd §43 stubs + /v1/compliance 501 stub → HRD-028.
HRD-095	task	low	bherald flavor scaffold — Build Herald CLI-only binary with 3 §43 stubs (HRD-035/041/045).
HRD-096	task	low	rherald flavor scaffold — Release Herald CLI-only binary with 3 §43 stubs (HRD-031/032/045).
HRD-097	task	low	iherald + scherald flavor scaffolds — Incident Herald serving :24794 with /v1/we → HRD-024 + Scheduled-audit Herald CLI-only with 1 §43 stub (HRD-047). Pa iherald has zero §43 stubs.
HRD-012	task	middle	Claude Code dispatcher live integration — <code>claude --resume <UUID> --p <envelope> exec + <<<HERALD-REPLY>>> JSON parse + claude_code_sessions persistence per §33. §107 evidence: Plan 2 Task 6 702b5a3 (live PASS 24.24s — real claude CLI round-trip, structured reply parse Outcome+Summary non-empty) + Task 7 commit 4718c0e (live PASS 36.23s — session_state upsert under HeraldSystemTenant with exact-equality assertions on session_uuid + anchor_path + last_response JSONB round-trip). HRD-085..HRD open for upstream-defined TaskRepository methods not exercised by the Dispatch path.</code>
HRD-010	task	middle	commons_storage live wiring — pgx pool + RLS-enforcing WithTenantContext (fixed RLS-bypass bug via E14 falsifiability) + 9 migrations + background queue (digital.vasic.background bound via pgxTaskRepository) + Redis ACL (digital.vapherald migrate up/status/down/validate subcommand + 3 new §107 e2e invariants E16) + HRD-085..090 registered for queue-repository stubs
HRD-080	task	low	Refine I6 inheritance-gate invariant from blanket <code>.gitmodules-forbidden to "constitution/entry in .gitmodules"</code> — paired meta-test <code>test_i6_refinement_meta.sh</code> with 3 anti-bluff subtests. Enables Foundation Helix-stack submodule installs.
HRD-017	task	middle	Propagate Universal §11.4.73 (main-spec versioning + revision discipline) and §1 (submodule-catalogue-first discovery) into parent constitution Constitution.md + CLAUDE.md + AGENTS.md
HRD-014	task	middle	pherald CLI scaffold — Cobra root + version + 5 stubbed subcommands; pherald <code>version --json</code> returns canonical build info
HRD-013	task	middle	commons_messaging + null:// adapter — full §11.0 Channel contract impl with ri fail_rate/latency/ceiling URL params + 8-case unit test suite
HRD-009b	task	low	commons_prefix module — §8.2 deterministic 3-letter prefix algorithm with Cam + collision-resolution via fnv1a32 + table-driven tests
HRD-009	task	middle	commons module — full §11.0 Go type contract (Channel + Capabilities + Deliver + OutboundMessage + Body + Attachment + Recipient + Action + Priority + Rec InboundHandler + InboundEvent + Subscriber + SubscriberAlias + CloudEventE TraceContext + Branding + ChannelID + PreferenceSet + ...) + Clock abstraction helper + DefaultBranding factory
HRD-007	task	middle	V3 r3 cross-doc sync + tracking-doc scaffold (Issues/Fixed/Status/Status_Summary Issues_Summary/Fixed_Summary/CONTINUATION)
HRD-006	task	middle	V3 r2 flavor refinement — 9 flavors × per-channel interaction tables
HRD-005	task	middle	V3 r1 operator-product layer (§31..§36 + §18.2 expansion)
HRD-004	task	middle	V2 r3 — Go type contract closure + operational ops detail
HRD-003	task	middle	V2 r2 — close prose [?] definition gaps + add operational guidance

ID	Type	Criticality	Title
HRD-002	task	middle	V2 r1 — architectural authoring (CloudEvents/Watermill/OTel/RLS/SLSA/9 flav
HRD-001	task	middle	V1 — initial MVP specification + Review section + Recommendations